

# Intrusion Detection Systems (IDS) and Traffic Analysis on Linux

## Introduction

This lab focused on setting up and configuring Snort as an Intrusion Detection System (IDS) on Kali Linux. Wireshark was used to capture and analyze network traffic, while hping3 generated a simulated SYN flood attack against the OWASP Broken Web Applications (BWA) virtual machine to evaluate Snort's detection capabilities.

## Objective

- Install and configure Snort on Kali Linux.
- Monitor network traffic to the OWASP BWA VM.
- Generate a SYN flood attack using hping3.
- Capture traffic with Wireshark.
- Detect and analyze malicious traffic using Snort.

## Materials Used

- Kali Linux
- OWASP Broken Web Applications VM
- Snort 3.12.2.0
- Wireshark
- hping3
- GNU Nano
- VMware Workstation
- Network Interface: eth0

## **Part 1: Prepare the Environment**

### **Step 1: Verify the OWASP Broken Web Applications VM is Running**

After ensuring that the OWASP Broken Web Applications VM was powered on tried pinging it using ping 192.168.5.130

#### **Error:**

The Kali Linux machine could not communicate with the OWASP Broken Web Applications.

### **Troubleshooting**

1. Identified that the VMs were on different subnets.
2. Changed the network adapters so both virtual machines used the same NAT network.

### **Step 2: Verify Connectivity**

ping 192.168.5.130 (owasp)

```
(kali@kali)-[~]  
$ ping 192.168.5.130  
PING 192.168.5.130 (192.168.5.130) 56(84) bytes of data.  
64 bytes from 192.168.5.130: icmp_seq=1 ttl=64 time=3.14 ms  
64 bytes from 192.168.5.130: icmp_seq=2 ttl=64 time=1.25 ms  
64 bytes from 192.168.5.130: icmp_seq=3 ttl=64 time=1.92 ms  
64 bytes from 192.168.5.130: icmp_seq=4 ttl=64 time=1.83 ms  
64 bytes from 192.168.5.130: icmp_seq=5 ttl=64 time=1.32 ms  
^C  
— 192.168.5.130 ping statistics —  
5 packets transmitted, 5 received, 0% packet loss, time 4014ms  
rtt min/avg/max/mdev = 1.254/1.892/3.143/0.679 ms
```

Network connectivity between the Kali Linux machine and the OWASP Broken Web Applications VM was verified successfully using the ping command. The successful replies confirmed that both systems could communicate over the network.

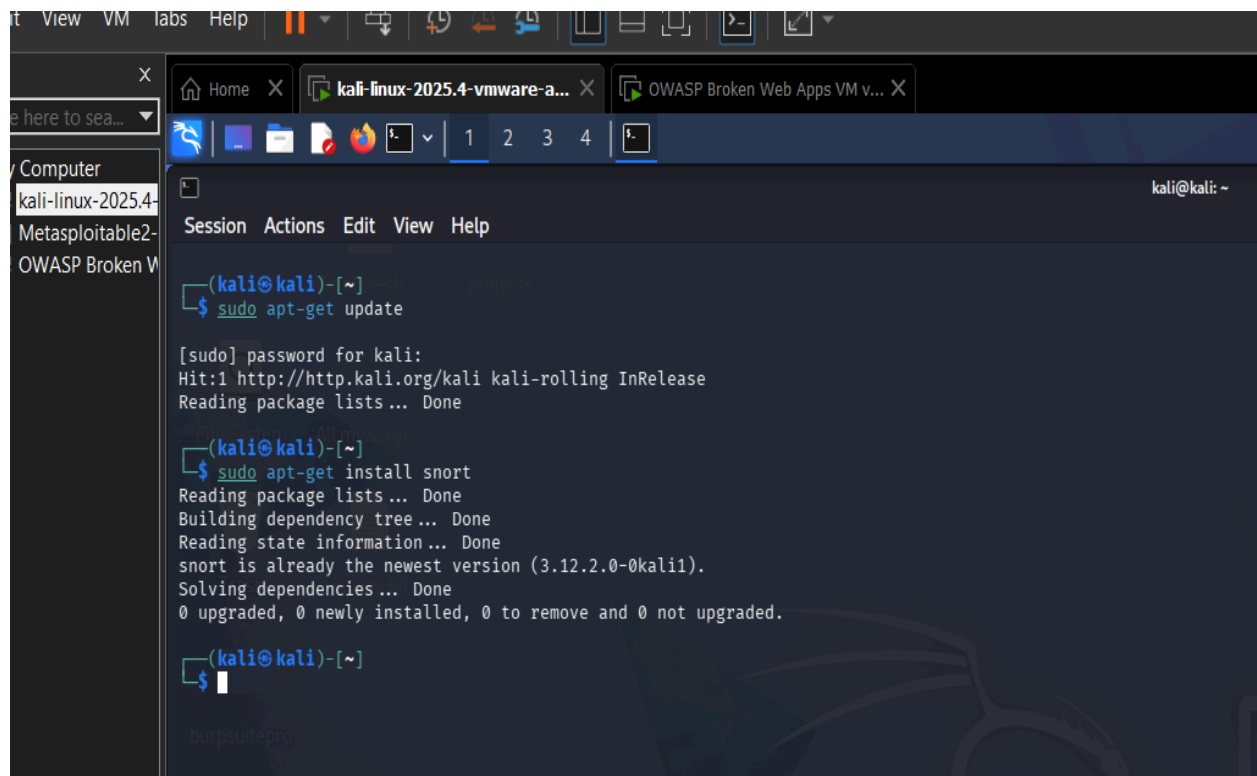
## Part 2: Install and Configure Snort

### Step 1: Install Snort

#### Commands used:

```
sudo apt-get update
```

```
sudo apt-get install snort
```



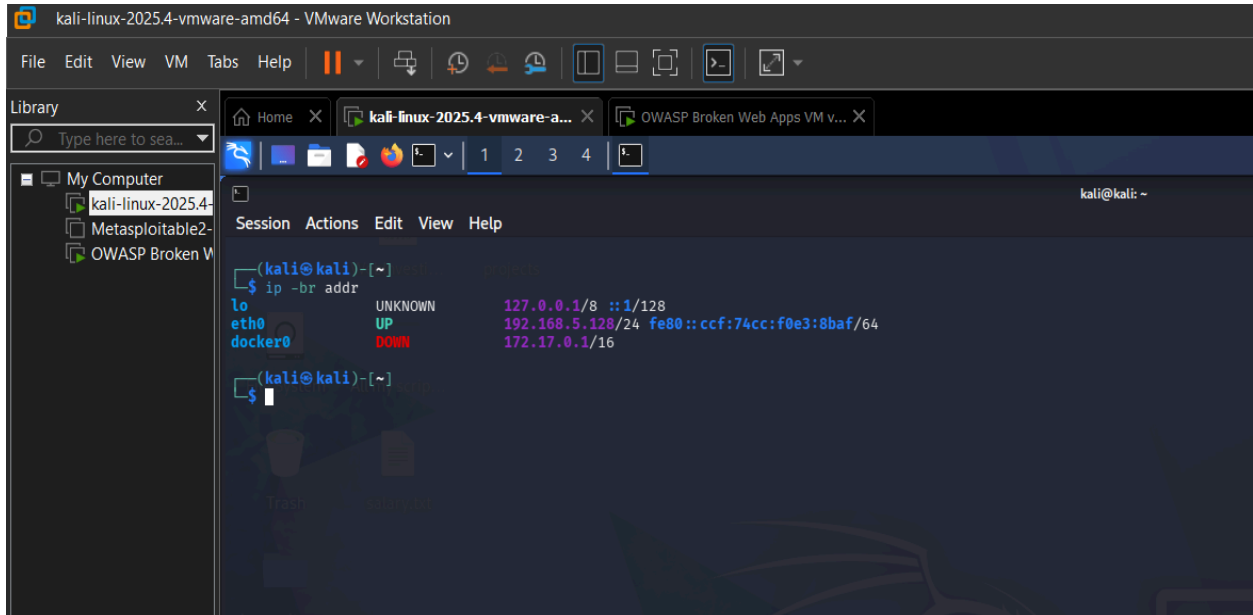
```
(kali@kali)-[~]  
$ sudo apt-get update  
  
[sudo] password for kali:  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
Reading package lists... Done  
  
(kali@kali)-[~]  
$ sudo apt-get install snort  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
snort is already the newest version (3.12.2.0-0kali1).  
Solving dependencies... Done  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.  
  
(kali@kali)-[~]  
$
```

Snort is already the newest version

## Step 2: Identify the Network Interface

### Command Used:

`ip -br addr`

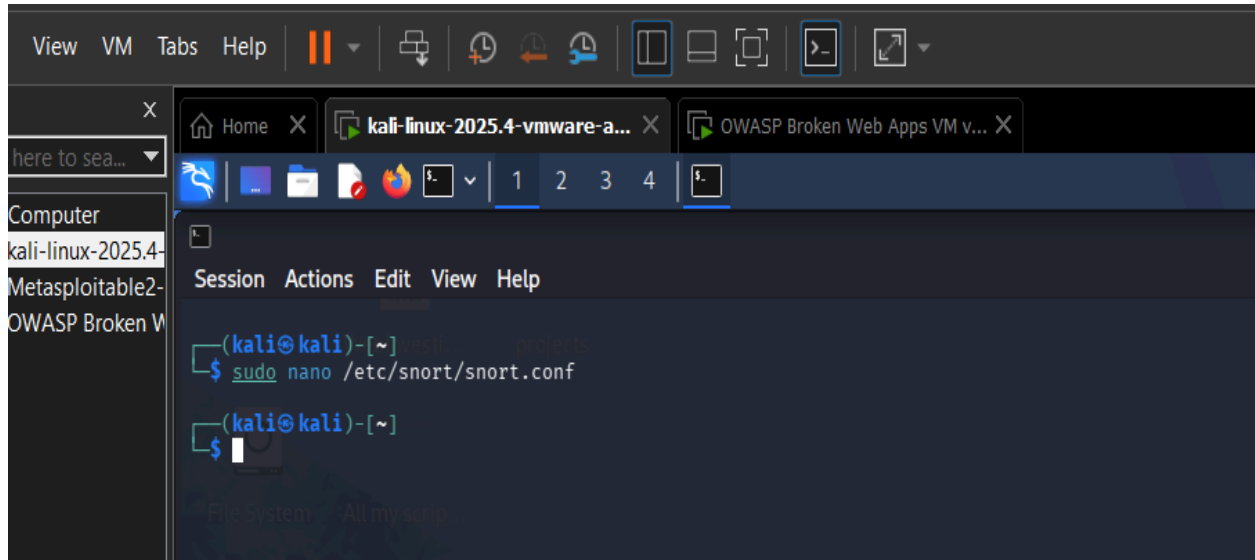


The active network interface was identified as **eth0**, with the IP address **192.168.5.128**. This interface will be used for Snort monitoring and Wireshark packet capture.

### Step 3: Configure Snort

**Command Used to open the nano file:**

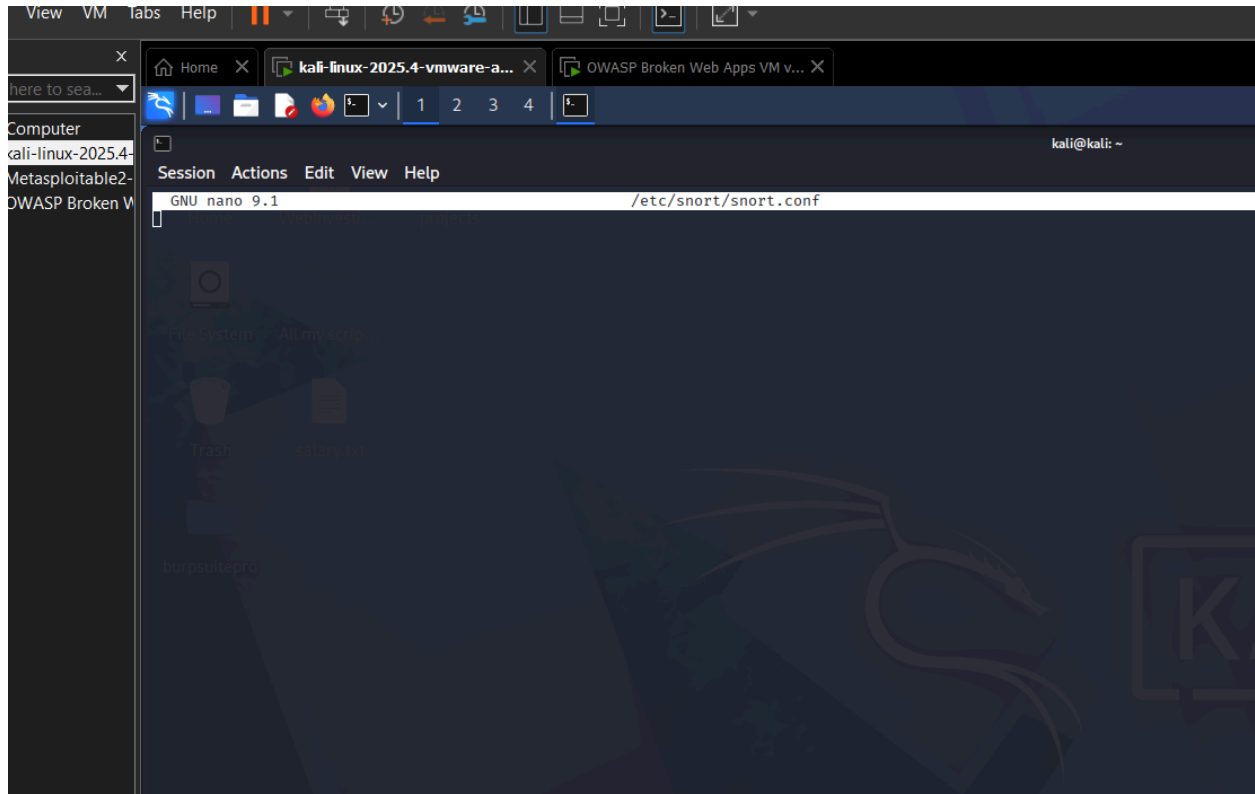
```
sudo nano /etc/snort/snort.conf
```



## Problem Encountered

While configuring Snort, the lab instructions directed me to edit the `/etc/snort/snort.conf` file. However, the file was empty, making it impossible to configure the `HOME_NET` variable as described in the lab manual.

## Empty File



## Troubleshooting Steps I Took:

### Verified the installed Snort version:

snort -V

1. The output showed that Snort 3.12.2.0 (Snort++) was installed.

### Listed the Snort configuration files:

ls -l /etc/snort

2. This revealed that the main configuration file was snort.lua instead of snort.conf.

### Located all Snort configuration files:

find /etc/snort -maxdepth 2 -type f

3. The command confirmed that Snort 3 uses **/etc/snort/snort.lua** and other Lua configuration files.

**NOT:** sudo nano /etc/snort/snort.conf

```
kali@kali: ~  
  
(kali@kali)-[~]  
$ snort -V  
  
-*> Snort++ <*-  
o" )~  
"  ~  
    ~  
Version 3.12.2.0  
By Martin Roesch & The Snort Team  
http://snort.org/contact#team  
Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.  
Copyright (C) 1998-2013 Sourcefire, Inc., et al.  
Using DAQ version 3.0.24  
Using libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)  
Using LuaJIT version 2.1.1761786044  
Using LZMA version 5.8.3  
Using OpenSSL 3.6.3 9 Jun 2026  
Using PCRE2 version 10.46 2025-08-27  
Using ZLIB version 1.3.2  
  
(kali@kali)-[~]  
$ ls -l /etc/snort  
total 248  
-rw-r--r-- 1 root root 590 Mar 17 21:38 balanced.lua  
-rw-r--r-- 1 root root 82469 Mar 20 06:10 community-sid-msg.map  
-rw-r--r-- 1 root root 616 Mar 17 21:38 connectivity.lua  
-rw-r--r-- 1 root root 45258 Mar 17 21:38 file_magic.rules  
-rw-r--r-- 1 root root 410 Mar 17 21:38 inline.lua  
-rw-r--r-- 1 root root 1123 Mar 17 21:38 max_detect.lua  
drwxr-xr-x 2 root root 12288 May 14 01:04 rules  
-rw-r--r-- 1 root root 814 Mar 17 21:38 security.lua  
-rw-r--r-- 1 root root 5426 Mar 17 21:38 sensitive_data.rules  
-rw-r--r-- 1 root root 400 Mar 20 06:10 snort.debian.lua  
-rw-r--r-- 1 root root 55688 Mar 17 21:38 snort_defaults.lua  
-rw-r--r-- 1 root root 9201 Apr 15 10:22 snort.lua  
-rw-r--r-- 1 root root 791 Mar 17 21:38 talos.lua  
  
(kali@kali)-[~]  
$ find /etc/snort -maxdepth 2 -type f  
/etc/snort/snort.debian.lua  
/etc/snort/connectivity.lua  
/etc/snort/balanced.lua  
/etc/snort/snort.lua  
/etc/snort/rules/web-attacks.rules  
/etc/snort/rules/virus.rules  
/etc/snort/rules/community-icmp.rules  
/etc/snort/rules/smtp.rules  
/etc/snort/rules/community-web-dos.rules
```

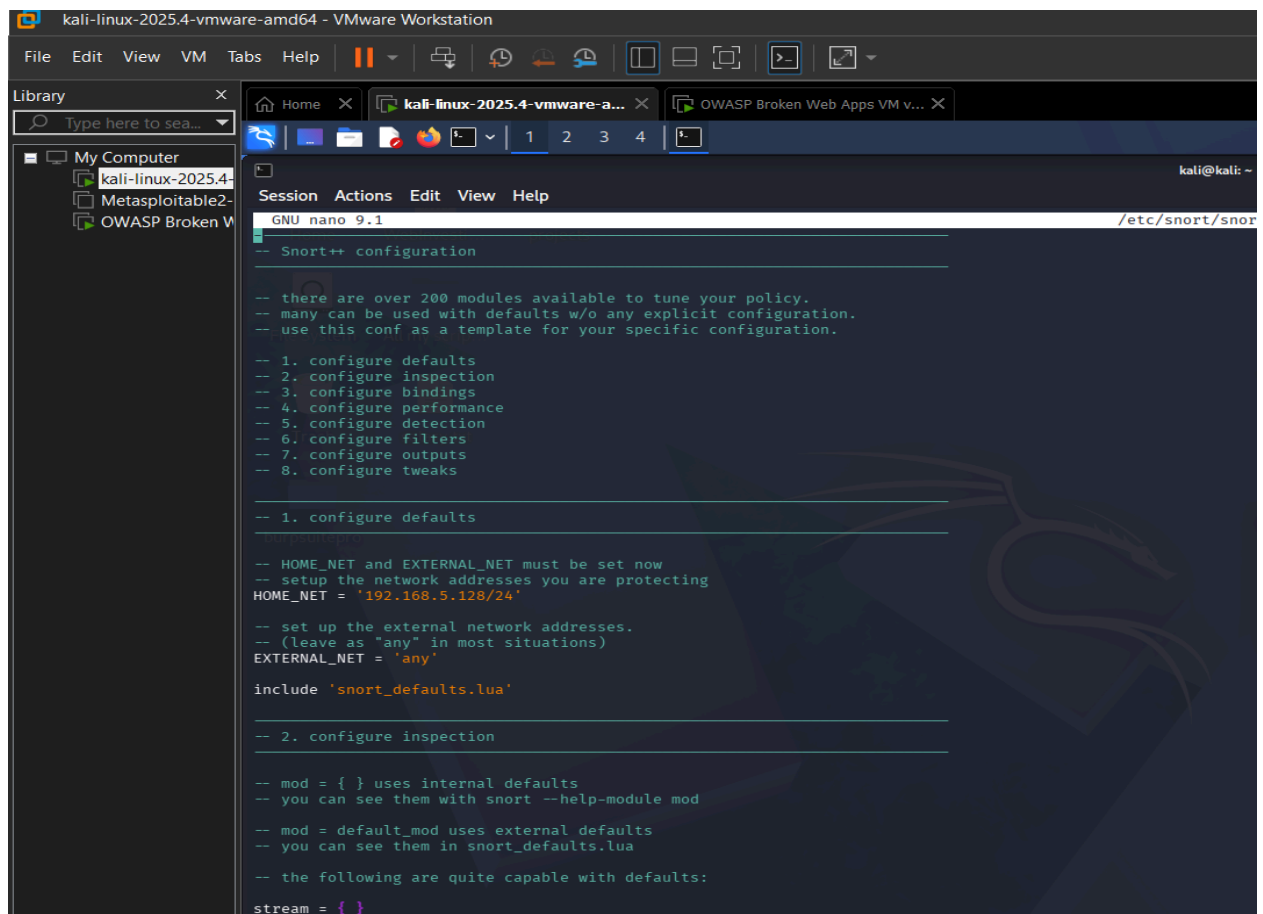
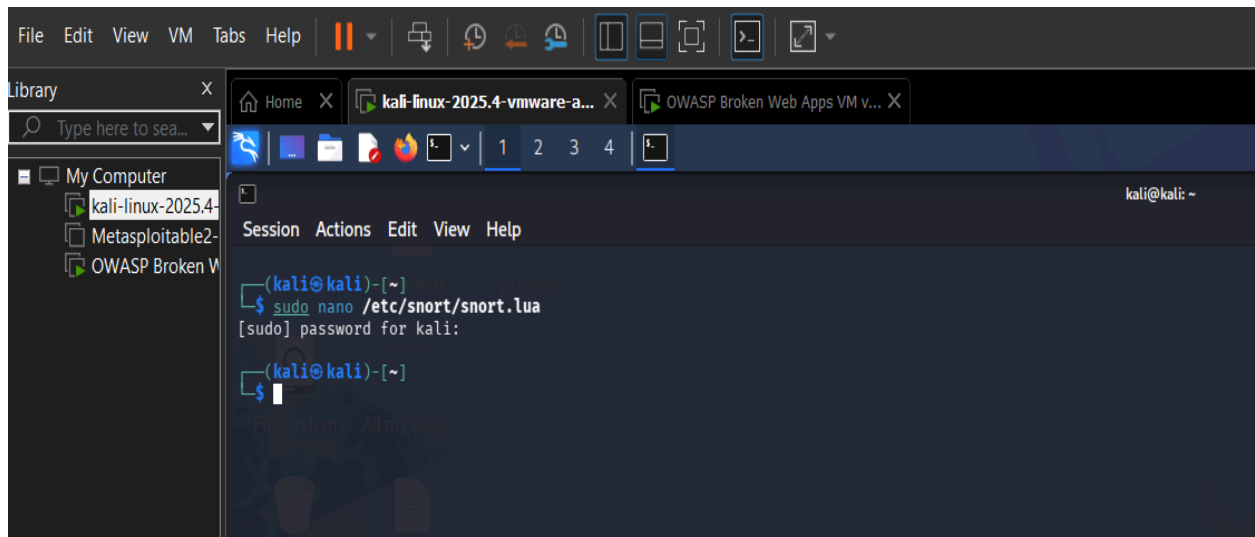
## Resolution

It was determined that the lab guide was written for Snort 2, while the Kali Linux system was running Snort 3. The configuration process was therefore adapted to use the Snort 3 configuration **file (snort.lua)** while still meeting the objectives of the lab.

## Proceed with Snort Configuration

## Command Used:

**sudo nano /etc/snort/snort.lua**

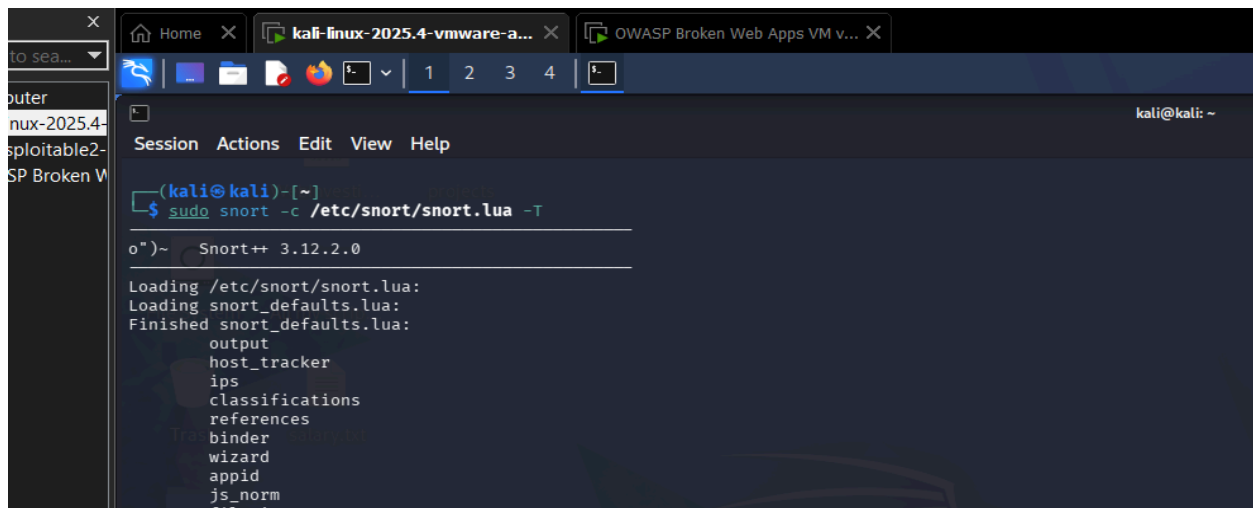


The Snort configuration file (snort.lua) was reviewed, and the HOME\_NET variable was confirmed to be correctly set to 192.168.5.128/24. This ensures that Snort monitors traffic within the protected network.

## Step 4: Validate the Snort Configuration

### Command:

```
sudo snort -c /etc/snort/snort.lua -T
```



The screenshot shows a terminal window with the following output:

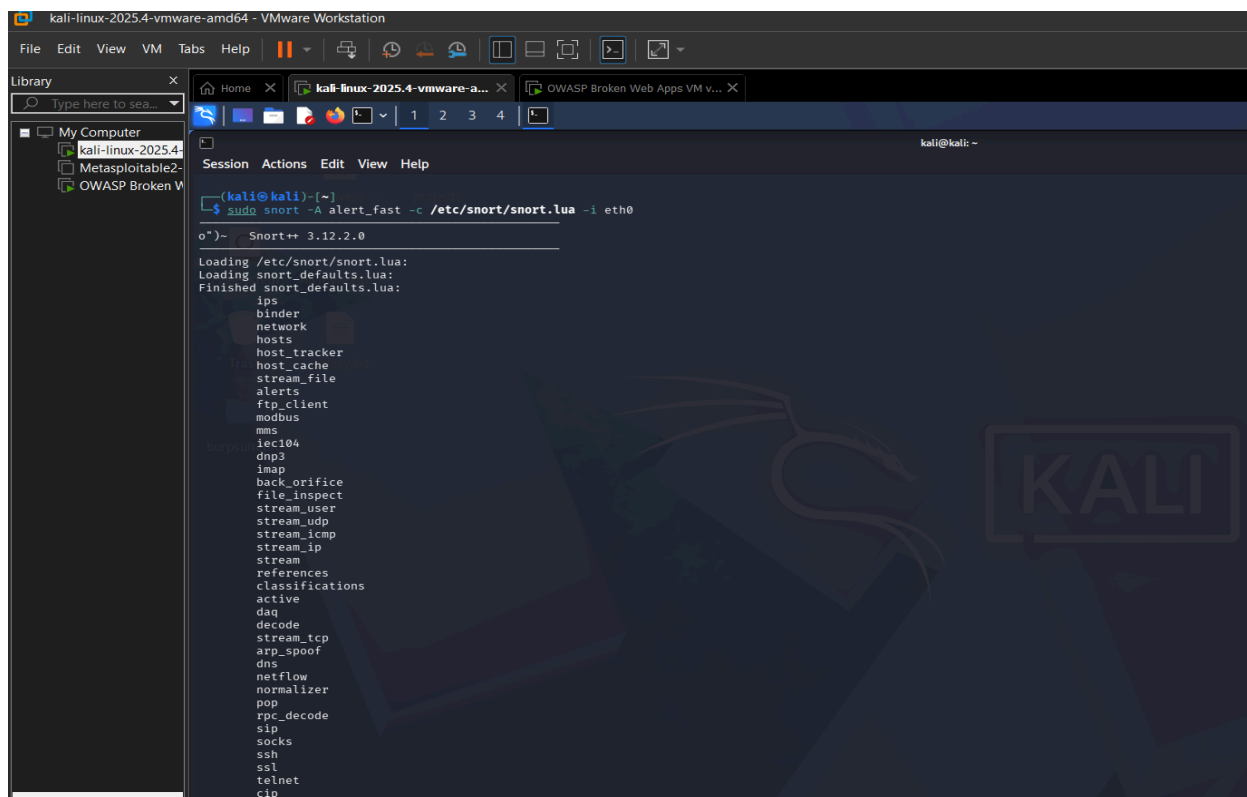
```
(kali@kali)-[~]  
$ sudo snort -c /etc/snort/snort.lua -T  
o*)~ Snort++ 3.12.2.0  
  
Loading /etc/snort/snort.lua:  
Loading snort_defaults.lua:  
Finished snort_defaults.lua:  
  output  
  host_tracker  
  ips  
  classifications  
  references  
  binder  
  wizard  
  appid  
  js_norm  
  file_inject
```

The Snort configuration was successfully validated using test mode. The validation completed with 0 warnings, confirming that the configuration file was correct and ready for IDS operation.

## Step 5: Start Snort in IDS Mode

### Command:

```
sudo snort -A alert_fast -c /etc/snort/snort.lua -i eth0
```



```

    wizard
    Max output (kilobits)      packets
    process
    search_engine
Finished /etc/snort/snort.lua:
Loading file_inspect.rules_file:
Loading file_magic.rules:
Finished file_magic.rules:
Finished file_inspect.rules_file:

```

---

```

ips policies rule stats
      id loaded  shared enabled  file
      1   219      0    219  /etc/snort/snort.lua

```

---

```

rule counts
  total rules loaded: 219
    text rules: 219
  option chains: 219
  chain headers: 1

```

---

```

service rule counts
      file_id: 219 219
      total:   219 219

```

---

```

fast pattern groups
  to_server: 1
  to_client: 1

```

---

```

search engine (ac_bnfa)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 2840
appid: patterns loaded: 300

```

---

```

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0

```

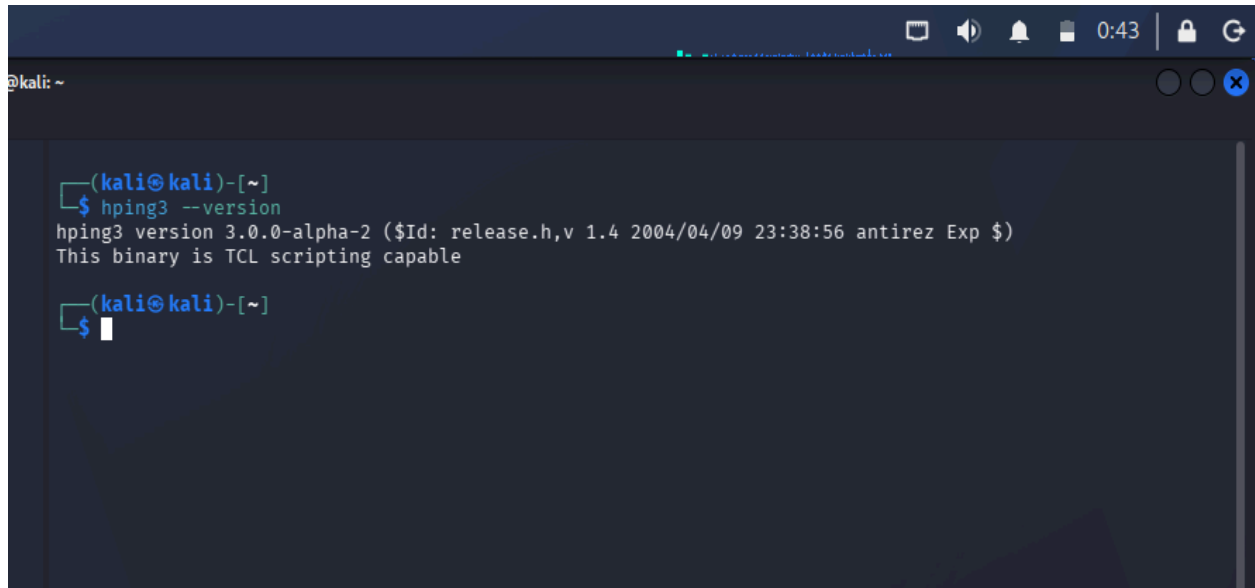
Snort was successfully started in IDS mode on the **eth0** network interface. The IDS began monitoring live network traffic and was ready to detect suspicious activity.

## Part 3: Generate Network Traffic

### Step 1: Verify hping3 is Installed

**Command Used:**

hping3 --version

A terminal window with a dark blue background and light blue text. The prompt is '(kali㉿kali)-[~]'. The command 'hping3 --version' has been entered, and the output is 'hping3 version 3.0.0-alpha-2 (\$Id: release.h,v 1.4 2004/04/09 23:38:56 antirez Exp \$)' followed by 'This binary is TCL scripting capable'. The prompt is now '\$' with a cursor.

```
(kali㉿kali)-[~]  
$ hping3 --version  
hping3 version 3.0.0-alpha-2 ($Id: release.h,v 1.4 2004/04/09 23:38:56 antirez Exp $)  
This binary is TCL scripting capable  
  
(kali㉿kali)-[~]  
$
```

The hping3 tool was verified to be installed successfully. It is available for generating custom network traffic to test Snort's intrusion detection capabilities.

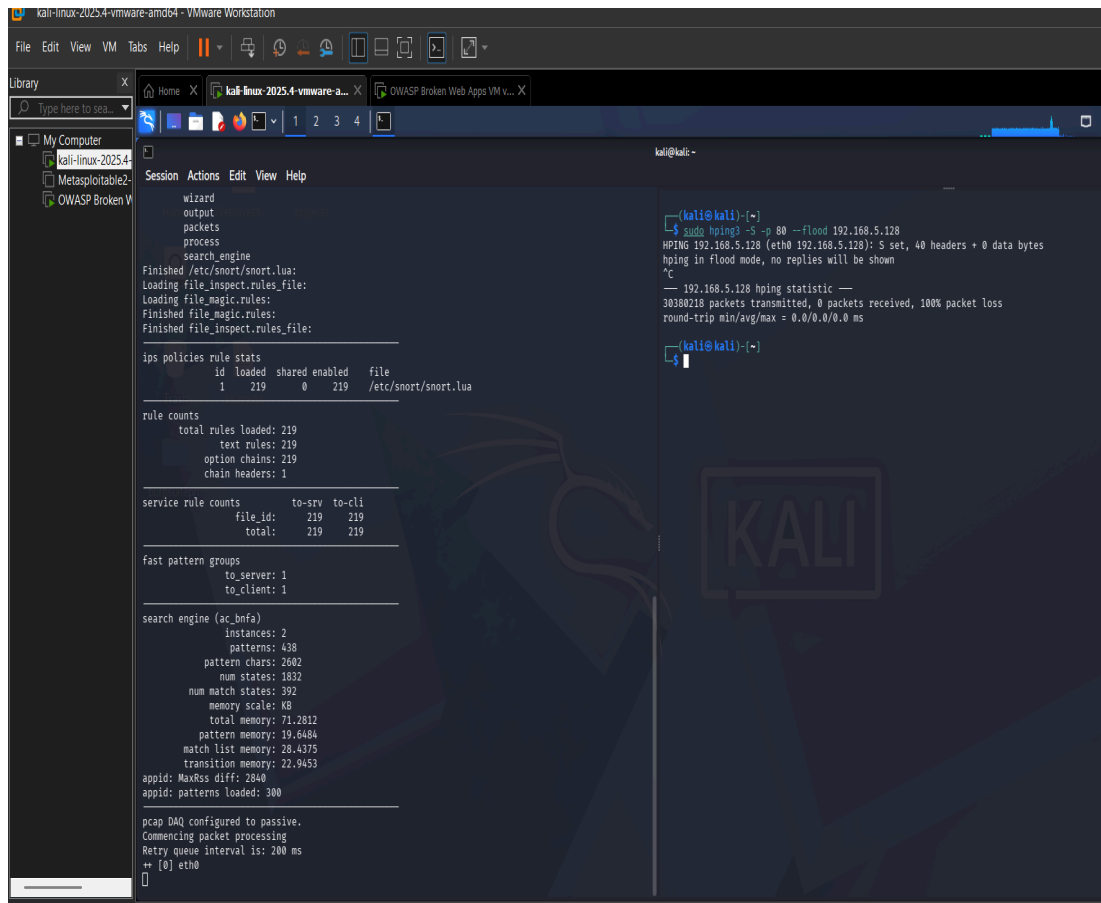
## Step 2: Generate a SYN Flood

### Command Used:

```
sudo snort -A alert_fast -c /etc/snort/snort.lua -i eth0
```

### On another terminal:

```
sudo hping3 -S -p 80 --flood 192.168.5.130
```



After generating the SYN flood traffic, no alerts were displayed in the Snort console. This indicates that no active Snort rule matched the generated traffic or that SYN flood detection rules were not enabled in the current configuration.

## Problem Encountered

No alerts were generated by Snort after the simulated SYN flood attack

## Troubleshooting

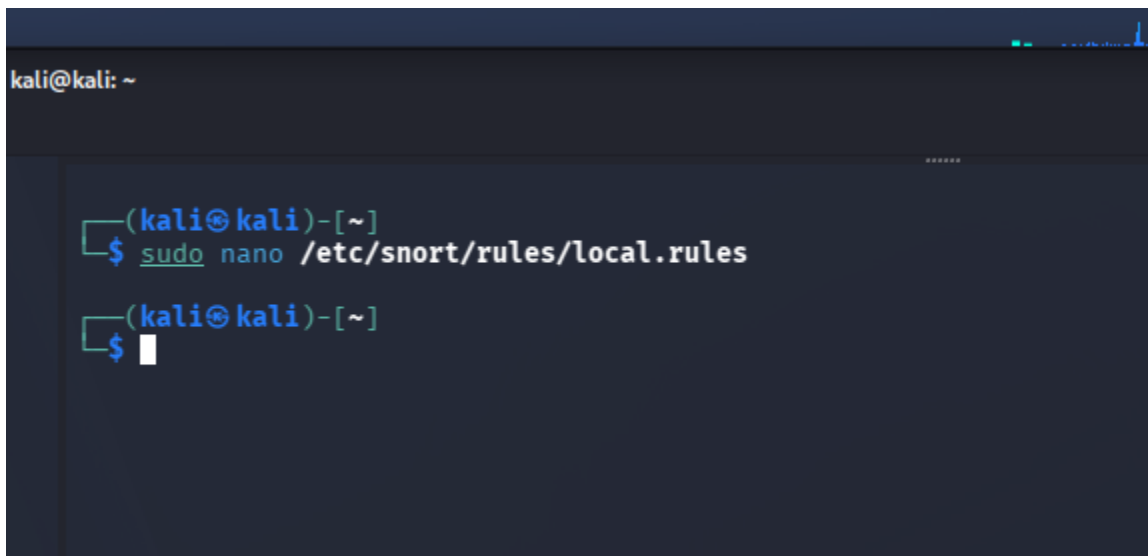
1. Confirmed that Snort was running in IDS mode on the correct interface (eth0).

2. Verified that hping3 successfully generated SYN flood traffic.
3. Determined that the installed Snort 3 configuration may not have an active rule to detect SYN flood traffic by default

**Verified by Opening the file using:**

**Command:**

`sudo nano /etc/snort/rules/local.rules`



```
kali@kali: ~  
  
(kali@kali)-[~]  
$ sudo nano /etc/snort/rules/local.rules  
  
(kali@kali)-[~]  
$
```



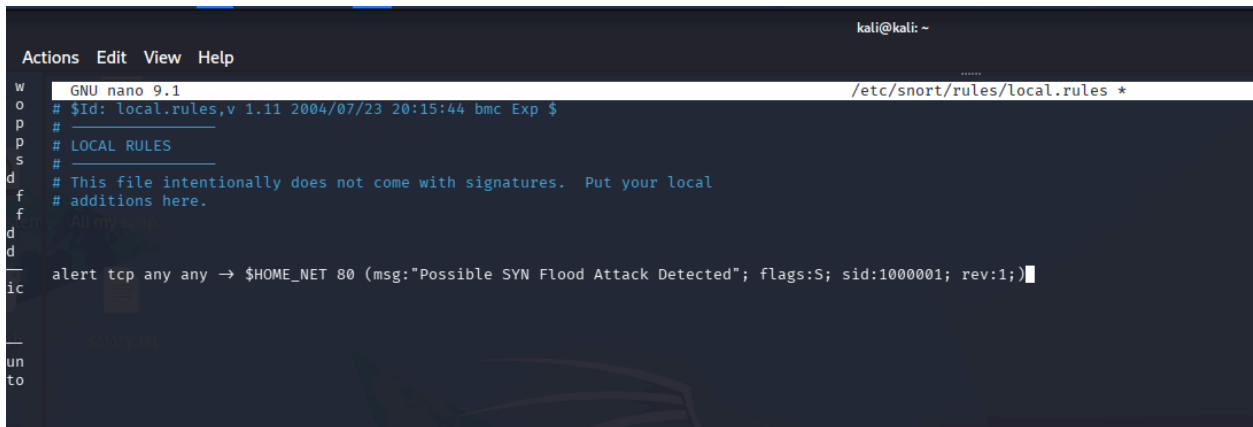
```
kali-linux-2025.4-vmware-a... X OWASP Broken Web Apps VM v... X  
1 2 3 4  
kali@kali: ~  
ns Edit View Help  
GNU nano 9.1 /etc/snort/rules/local.rules  
$Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
LOCAL RULES  
This file intentionally does not come with signatures. Put your local  
additions here.  
All my work  
S2025-04
```

## Discovered:

The reason no alerts appeared is that the file is essentially empty with no rule.

## So I added a simple local rule specifically for this lab.

alert tcp any any -> \$HOME\_NET 80 (msg:"Possible SYN Flood Attack Detected"; flags:S; sid:1000001; rev:1;)



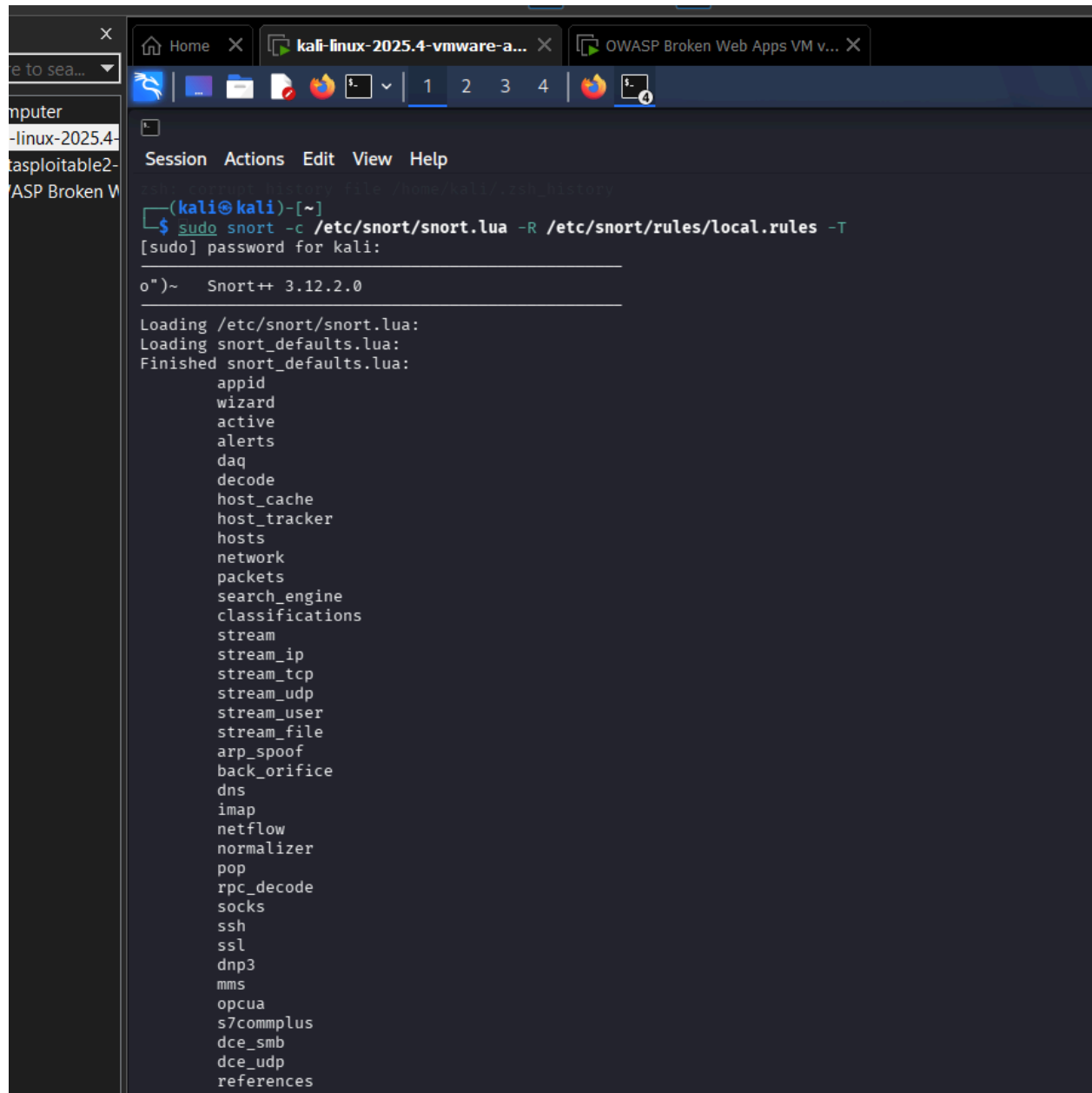
```
kali@kali: ~  
Actions Edit View Help  
GNU nano 9.1 /etc/snort/rules/local.rules *  
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
#  
# LOCAL RULES  
#  
# This file intentionally does not come with signatures. Put your local  
# additions here.  
  
alert tcp any any -> $HOME_NET 80 (msg:"Possible SYN Flood Attack Detected"; flags:S; sid:1000001; rev:1;)
```

Saved and exit:

- Ctrl + O
- Enter
- Ctrl + X

## Validate the rule file:

```
sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -T
```



```
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -T
[sudo] password for kali:
o")~ Snort++ 3.12.2.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  appid
  wizard
  active
  alerts
  daq
  decode
  host_cache
  host_tracker
  hosts
  network
  packets
  search_engine
  classifications
  stream
  stream_ip
  stream_tcp
  stream_udp
  stream_user
  stream_file
  arp_spoof
  back_orifice
  dns
  imap
  netflow
  normalizer
  pop
  rpc_decode
  socks
  ssh
  ssl
  dnp3
  mms
  opcua
  s7commplus
  dce_smb
  dce_udp
  references
```

## **Run Snort again:**

### **command used:**

```
sudo snort -A alert_fast -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -i eth0
```

### **On another terminal:**

```
sudo hping3 -S -p 80 --flood 192.168.5.130
```

```
VMware Workstation
View VM Tabs Help
kali-linux-2025.4-vmware-a... X
OWASP Broken Web Apps VM v... X
kali@kali: ~
Session Actions Edit View Help
kali@kali:~$ sudo snort -A alert_fast -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -i eth0
[sudo] password for kali:
o*)~ Snort++ 3.12.2.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  active
  alerts
  daq
  decode
  host_cache
  host_tracker
  hosts
  network
  process
  search_engine
  arp_spoof
  back_orifice
  imap
  netflow
  normalizer
  pop
  rpc_decode
  socks
  appid
  wizard
  binder
  references
  classifications
  output
  http_inspect
  ips
  js_norm
  file_policy
  file_inspect
  http2_inspect
  ftp_data
  ftp_client
  ftp_server
  smtp
  port_scan
  gtp_inspect
  dce_http_server

(kali@kali)~$ sudo hping3 -S -p 80 --flood 192.168.5.130
HPING 192.168.5.130 (eth0 192.168.5.130): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.5.130 hping statistic ---
44126 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms

(kali@kali)~$
```

```

kali@kali: ~
Session Actions Edit View Help

prcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0
07/14-04:02:18.968996 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1507 → 192.168.5.130:80
07/14-04:02:18.969229 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1508 → 192.168.5.130:80
07/14-04:02:18.969309 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1509 → 192.168.5.130:80
07/14-04:02:18.969382 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1510 → 192.168.5.130:80
07/14-04:02:18.969545 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1511 → 192.168.5.130:80
07/14-04:02:18.969623 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1512 → 192.168.5.130:80
07/14-04:02:18.969779 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1513 → 192.168.5.130:80
07/14-04:02:18.969869 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1514 → 192.168.5.130:80
07/14-04:02:18.970046 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1515 → 192.168.5.130:80
07/14-04:02:18.970124 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1516 → 192.168.5.130:80
07/14-04:02:18.970201 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1517 → 192.168.5.130:80
07/14-04:02:18.970279 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1518 → 192.168.5.130:80
07/14-04:02:18.970501 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1519 → 192.168.5.130:80
07/14-04:02:18.970588 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1520 → 192.168.5.130:80
07/14-04:02:18.970668 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1521 → 192.168.5.130:80
07/14-04:02:18.970905 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1522 → 192.168.5.130:80
07/14-04:02:18.970907 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1523 → 192.168.5.130:80
07/14-04:02:18.970908 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1524 → 192.168.5.130:80
07/14-04:02:18.970908 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1525 → 192.168.5.130:80
07/14-04:02:18.971014 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1526 → 192.168.5.130:80
07/14-04:02:18.971015 [**] [1:1000001:1] "Possible SYN Flood Attack Detected" [**] [Priority: 0]
{TCP} 192.168.5.128:1527 → 192.168.5.130:80

kali@kali: ~
(kali@kali)-[~]
└─$ sudo hping3 -S -p 80 --flood 192.168.5.130
HPING 192.168.5.130 (eth0 192.168.5.130): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.5.130 hping statistic —
44126 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms

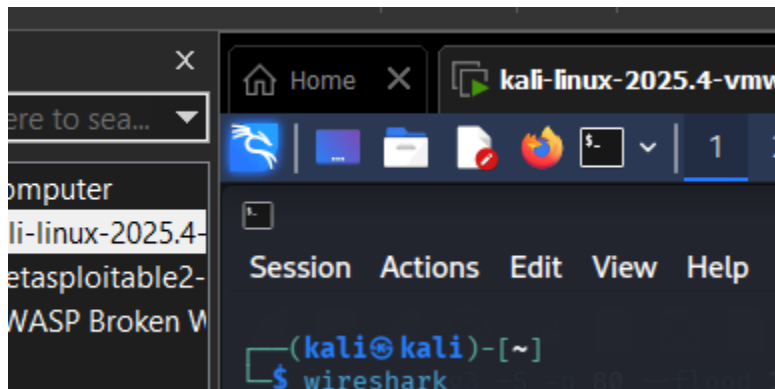
kali@kali: ~
└─$
```

Snort was successfully configured to monitor network traffic on the eth0 interface. A custom detection rule stored in local.rules was loaded using the -R option. During the simulated SYN flood attack generated with hping3, Snort detected multiple TCP SYN packets directed to the OWASP Broken Web Applications VM on port 80 and generated alerts indicating a possible SYN flood attack.

## Part 4: Monitor Traffic with Wireshark

### Step 1: Start Wireshark

Command:  
wireshark



## Step 2: Selected my network Interface

eth0

## Step 3: Applied a Display Filter

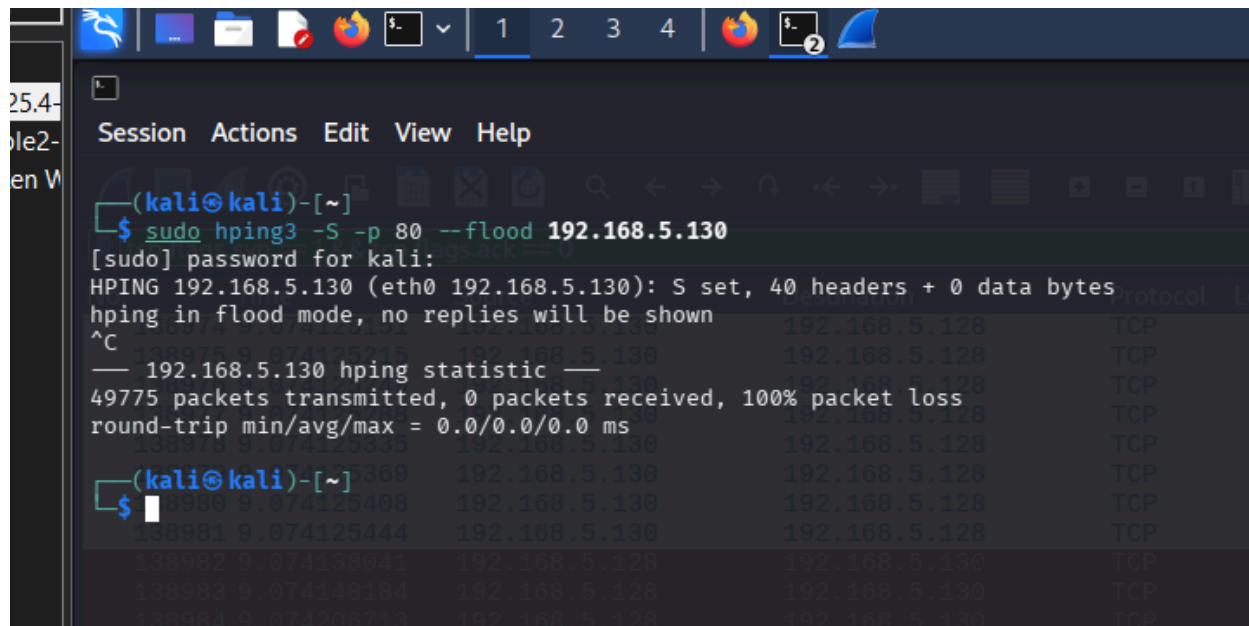
```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

(This showed only the initial SYN packets.)

## Step 4: Generated the Attack

In another terminal:

```
sudo hping3 -S -p 80 --flood 192.168.5.130
```



The screenshot shows a Kali Linux terminal window with a dark theme. The terminal displays the command `sudo hping3 -S -p 80 --flood 192.168.5.130` being executed. The output shows that the command was successful, with 49775 packets transmitted and 0 packets received, resulting in 100% packet loss. The terminal also shows the hping3 statistics for the target IP address 192.168.5.130.

```
(kali㉿kali)-[~]  
$ sudo hping3 -S -p 80 --flood 192.168.5.130  
[sudo] password for kali:  
HPING 192.168.5.130 (eth0 192.168.5.130): S set, 40 headers + 0 data bytes  
hping in flood mode, no replies will be shown  
^C  
— 192.168.5.130 hping statistic —  
49775 packets transmitted, 0 packets received, 100% packet loss  
round-trip min/avg/max = 0.0/0.0/0.0 ms  
  
(kali㉿kali)-[~]  
$
```

| No.    | Time        | Source        | Destination   | Protocol | Length | Info                                                      |
|--------|-------------|---------------|---------------|----------|--------|-----------------------------------------------------------|
| 138974 | 9.074125151 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49453 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138975 | 9.074125215 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49454 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138976 | 9.074125249 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49455 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138977 | 9.074125288 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49456 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138978 | 9.074125335 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49457 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138979 | 9.074125369 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49458 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138980 | 9.074125408 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49459 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138981 | 9.074125444 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49460 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138982 | 9.074138041 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49453 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138983 | 9.074148184 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49454 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138984 | 9.074206713 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49455 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138985 | 9.074218088 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49456 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138986 | 9.074276797 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49457 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138987 | 9.074288077 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49458 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138988 | 9.074355105 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49459 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138989 | 9.074365490 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49460 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138990 | 9.074429547 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49461 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138991 | 9.074429640 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49462 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138992 | 9.074429712 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49463 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138993 | 9.074429768 | 192.168.5.130 | 192.168.5.128 | TCP      | 60     | 80 → 49464 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 |
| 138994 | 9.074436904 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49461 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138995 | 9.074447327 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49462 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138996 | 9.074503839 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49463 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138997 | 9.074514966 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49464 → 80 [RST] Seq=1 Win=0 Len=0                        |
| 138998 | 9.074619425 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49701 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 138999 | 9.074635746 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49702 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 139000 | 9.074721061 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49703 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 139001 | 9.074737641 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49704 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 139002 | 9.074832163 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49705 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 139003 | 9.074847082 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49706 → 80 [SYN] Seq=0 Win=512 Len=0                      |
| 139004 | 9.074938572 | 192.168.5.128 | 192.168.5.130 | TCP      | 54     | 49707 → 80 [SYN] Seq=0 Win=512 Len=0                      |

▶ Frame 1: Packet, 93 bytes on wire (744 bits), 93 bytes captured (744 bits) on interface eth0  
 ▶ Ethernet II, Src: VMware 4b:12:cf (00:0c:29:4b:12:cf), Dst: VMware e2:35:4d (00:50:56:e2:35:4d)  
 ▶ Internet Protocol Version 4, Src: 192.168.5.128, Dst: 34.107.243.93  
 ▶ Transmission Control Protocol, Src Port: 57186, Dst Port: 443, Seq: 1, Ack: 1, Len: 39  
 ▶ Transport Layer Security

Wireshark successfully captured the TCP SYN packets generated by the hping3 tool. The packet capture showed repeated TCP SYN requests originating from the Kali Linux machine (192.168.5.128) and targeting the OWASP Broken Web Applications VM (192.168.5.130) on port 80. The captured traffic confirmed that the simulated SYN flood attack reached the destination and matched the alerts generated by Snort.

## Summary of Challenges and Troubleshooting

The main challenges encountered were the use of Snort 3 instead of Snort 2, the failure of Snort to load the custom detection rule, and network communication issues between the Kali and OWASP virtual machines. These issues were

resolved by using the correct Snort configuration file (snort.lua), loading the custom rule with the -R option, and configuring both virtual machines on the same NAT network.

## **Conclusion**

The lab was successfully completed by configuring Snort to monitor network traffic and detect a simulated SYN flood attack. Wireshark captured the attack traffic, while Snort generated alerts, demonstrating the effectiveness of an IDS in detecting suspicious network activity.